



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 wide notification number F-9-12/2001-U-3 of Government of India)

॥वसुधैव कुटुम्बकम्॥

Re-Accredited by NAAC with 'A++' Grade | Website: www.siu.edu.in | www.sitnagpur.edu.in



Celebrating 50 Years of Excellence

Course: Data Compression

Course Code: 705210502

RLE Practices

Dr. Gyana Ranjana Panigrahi

each run stored as (symbol: 1 byte, count: 1 byte). Original text uses 1 byte/char

Text RLE: clear savings

Input: AAABBBAA

Runs: A3, B3, A2 → 3 runs

Original size

$$= 8 \text{ chars} \times 1 = \mathbf{8 \text{ bytes}}$$

Compressed size

$$= 3 \text{ runs} \times (1 + 1) = \mathbf{6 \text{ bytes}}$$

Compression Ratio (CR)

$$= \frac{6}{8} = \mathbf{0.75} \rightarrow 25\% \text{ reduction}$$

Storage assumption

Let's assume:

1 byte for symbol (value)

1 byte for count (run length)

Total = 2 bytes per run

If you have 3 runs, then:

$$\text{Compressed size} = \text{Number of runs} \times (\text{symbol size} + \text{count size})$$

$$= 3 \times (1 + 1)$$

$$= 3 \times 2 = 6 \text{ bytes}$$

Text RLE: mixed runs

Input: **WWWWBWWWWWWBBBWWWW** (length = 18)

Runs: **W4, B1, W6, B3, W4** → 5 runs

Original

= 18 bytes

Compressed

= $5 \times 2 = 10$ bytes

CR

= $\frac{10}{18} \approx 0.556$

Text RLE: worst case expansion

Input: ABCDEF (all singletons)

Runs: A1, B1, C1, D1, E1, F1 → 6 runs

Original

= 6 bytes

Compressed

= $6 \times 2 = 12$ bytes

CR

= $\frac{12}{6} = 2.0 \rightarrow 100\%$ larger (inefficient)

If **average run length** > 2, RLE saves space (compression).

If **average run length** = 2, RLE neither helps nor hurts.

If **average run length** < 2 (like ABAB pattern, run length = 1), RLE actually **expands** the data.

RLE is great when you have **long stretches** of the same symbol (e.g., AAAAAAA).

But it's terrible when symbols change often (ABABAB...), because instead of storing just 1 byte per symbol, you now need **2 bytes**.

1-bit-looking row, but stored as bytes

Row (15 pixels): 000000111110000

Runs: 0×6, 1×5, 0×4 → 3 runs

Original (byte/pixel)

= $15 \times 1 = 15$ bytes

Compressed (value 1B + count 1B)

= $3 \times 2 = 6$ bytes

CR

= $\frac{6}{15} = 0.4 \rightarrow 60\%$ reduction

5

8-bit grayscale row

Pixels: 100 100 100 255 255 0 0 0 0 (9 pixels)

Runs: 100×3, 255×2, 0×4 → 3 runs

Original

= $9 \times 1 = 9$ bytes

Compressed

= $3 \times 2 = 6$ bytes

CR

= $\frac{6}{9} \approx 0.667 \rightarrow 33.3\%$ reduction



Various Bit Depth Representation



2-bit



4-bit



6-bit



8-bit



10-bit

Count limit (255) forces split

Pixels: a run of 300 zeros (8-bit grayscale)

Runs after splitting: 0×255 , $0 \times 45 \rightarrow 2$ runs

Original

$$= 300 \times 1 = \mathbf{300 \text{ bytes}}$$

Compressed (value+count each run)

$$= 2 \times 2 = \mathbf{4 \text{ bytes}}$$

CR

$$= \frac{4}{300} \approx \mathbf{0.0133} \rightarrow \mathbf{98.7\% \text{ reduction}}$$

Mixed long/short runs (text model)

String length: 120 chars

Run counts (in order): [20, 1, 1, 15, 30, 2, 2, 25, 4] → 9 runs

(Check sum: $20+1+1+15+30+2+2+25+4 = 100$; add 20 more singletons at end to reach 120 total → that's 20 extra runs. Total runs = $9 + 20 = 29$ runs)

Original

= 120 bytes

Compressed (1B symbol + 1B count)

= $29 \times 2 = 58$ bytes

CR

= $\frac{58}{120} \approx 0.483 \rightarrow 51.7\%$ reduction

Check sum a quick check that the runs add up to the total string length

20 characters are still missing → those are the extra singletons at the end.

Insight: Many singletons hurt compression, but long runs (20, 15, 30, 25) still keep CR < 0.5.

Why this makes sense

The **long runs** (20, 15, 30, 25) compress very well.

Example: AAAA... (30 times) → stored as just 2 bytes instead of 30!

The **short runs** (like 1, 2, 4, and the 20 singletons) are bad for RLE.

Example: a single character x → stored as $(x, 1) = 2$ bytes instead of 1 (expansion!).

But overall, since we have **several big runs**, the compression ratio is still **good** (0.483).

That's why the text says:

Many singletons hurt compression, but long runs (20, 15, 30, 25) still keep $CR < 0.5$.

Two RLE variants compared (same data)

Run counts: [8, 12, 3, 1, 16, 7, 2, 14] (sum = 63 symbols)

Assume runs alternate symbols (e.g., A/B/A/...).

Variant A (byte/byte): (symbol:1B, count:1B)

Runs = 8 $\rightarrow$ size = $8 \times 2 = \mathbf{16 \text{ bytes}}$

Original size = 63 bytes

$$CR_A = \frac{16}{63} \approx \mathbf{0.254}$$

Variant B (nibble count, max 15): store (symbol:1B, count:4 bits).

Any run with count > 15 must split. Here only 12, 8, 3, 1, 7, 2, 14 ≤ 15 ; 16 splits into 15 + 1 (adds one run).

Effective runs = 9 (because 16 $\rightarrow$ two runs)

Size (pack two counts per byte):

Symbols: $9 \times 1B = 9 \text{ bytes}$

Counts: 9 nibbles = 4.5 bytes $\rightarrow$ stored as 5 bytes (byte-aligned)

Total = 14 bytes

$$CR_B = \frac{14}{63} \approx \mathbf{0.222} \rightarrow \text{Variant B wins (fewer count bits)}$$

2 nibbles = 1 byte

1 nibble = 0.5 byte

9 nibbles = $9 \times 0.5 \text{ byte} = 4.5 \text{ bytes}$

Cont.

Run counts:

[8, 12, 3, 1, 16, 7, 2, 14]

This means we have **8 runs total**, and the runs alternate symbols (say A, B, A, B, ...).

Check sum:

$$8 + 12 + 3 + 1 + 16 + 7 + 2 + 14 = 63 \quad \text{symbols total.}$$

So the **original string length = 63 bytes**.

Variant A — Simple RLE (1 byte for symbol, 1 byte for count)

Each run is stored as (symbol, count) = 2 bytes.

We have 8 runs.

So:

$$CR = \frac{\text{Compressed Size}}{\text{Original Size}}$$

$$8 \times 2 = 16 \text{ bytes}$$

$$CR_A = \frac{16}{63}$$

So Variant A keeps only 25.4% of the original size, meaning it saves about 74.6%

Variant B — Optimized RLE (1 byte symbol + 4-bit count)

Here we try to save space by storing the count in only 4 bits (a nibble), instead of a full byte.

A nibble can represent 0–15.

If a run's count > 15, it must be split into multiple runs.

Let's check runs:

8 (≤ 15 , fine)

12

3

1

16 → must split into (15 + 1) → that becomes **two runs**.

7

2

14

So instead of 8 runs, we now have $8 + 1 = 9$ runs (because 16 split into 2).

Storage calculation for Variant B

1. Symbols:

Each run has a symbol (1 byte).

$$9 \times 1 = 9 \text{ bytes.}$$

2. Counts:

Each run count is a nibble (4 bits).

So 9 counts = 9 nibbles = 4.5 bytes.

Since storage is **byte-aligned**, we must round up → 5 bytes.

3. Total size = symbols + counts =

$$9 + 5 = 14 \text{ bytes.}$$

Cont.

Compression ratio (CR_B)

$$CR_B = \frac{14}{63} \approx 0.222$$

So Variant B uses less space than Variant A (14 vs 16 bytes) → better compression.

Why Variant B is good:

Variant A wastes space because every count always takes 1 full byte, even if the run is tiny.

Variant B uses only 4 bits per count, almost halving count storage.

Splitting runs bigger than 15 adds slight overhead, but here it's small compared to the savings.

That's why Variant B compresses to 22.2% of original size, vs Variant A's 25.4%.

So the key difference:

Variant A = simpler, but wastes bits.

Variant B = more compact, but must handle run splitting.

Cont.

Why Variant B uses nibble (4 bits) for the count?

In normal RLE (Variant A), for each run we store:

- Symbol = 1 byte (8 bits)
- Count = 1 byte (8 bits)
→ So every run uses **2 full bytes**.

But in some data (like images, simple text), **run counts are usually small** (e.g., 1 to 15 repeated characters).

That means we're wasting space by always giving **8 bits (0–255)** to store a count when often we only need a small number.

Optimization idea → Store count in nibbles (4 bits)

A **nibble** = half a byte = 4 bits = can store values from **0 to 15**.

If we use **1 byte for symbol + 4 bits for count**, then:

- We save space whenever run counts are small (≤ 15).
- But if a run is larger than 15 (e.g., 16, 20, 30, etc.), we **split the run** into multiple chunks (like 15 + remainder).

Example from your problem

Runs = [8, 12, 3, 1, 16, 7, 2, 14]

All counts ≤ 15 except **16**, which must split $\rightarrow 15 + 1$.

So runs become **9 runs** total instead of 8.

How storage is done in Variant B

Symbols: Each run still needs **1 byte** (so 9 bytes).

Counts: Each run uses **4 bits**. Two counts can be packed into 1 byte.

9 runs $\rightarrow$ 9 counts $\rightarrow$ 9 nibbles = **4.5 bytes** $\rightarrow$ stored as **5 bytes**.

So total storage = **9 + 5 = 14 bytes**.

Much smaller than Variant A (16 bytes).

Worst-case alternating pattern (expansion analysis)

Input: length $N = 100$ with strict alternation, e.g., ABABAB...

Runs: every run has count = 1 $\rightarrow$ 100 runs

Original (1 byte/char)

= 100 bytes

Compressed (byte/byte model)

= $100 \times 2 = 200$ bytes

CR

= $\frac{200}{100} = 2.0 \rightarrow 100\%$ larger

If average run length > 2 , RLE saves space (compression).

If average run length = 2, RLE neither helps nor hurts.

If average run length < 2 (like ABAB pattern, run length = 1), RLE actually expands the data.

RLE is great when you have long stretches of the same symbol (e.g., AAAAAAA).

But it's terrible when symbols change often (ABABAB...), because instead of storing just 1 byte per symbol, you now need 2 bytes.



॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 wide notification number F-9-12/2001-U-3 of Government of India)

Re-Accredited by NAAC with 'A++' Grade | Website: www.siu.edu.in | www.sitnagpur.edu.in



Celebrating 50 Years of Excellence

Thank You